Aerial Guardian – Drone Based Multi Object Tracking

This report describes the final aerial multi-object tracking pipeline developed for drone-based surveillance scenarios. The project focuses on person detection and multi-object tracking in aerial imagery where objects are extremely small, camera motion is dynamic, and maintaining stable identities is difficult.

1. Final Architecture Used

The final implementation uses a lightweight real-time architecture based on:

- YOLOv8s for person detection
- ByteTrack for multi-object tracking
- Image-sequence based inference using VisDrone MOT sequences
- Dynamic confidence thresholding
- Trajectory smoothing
- Detection interval scheduling
- Minimum box-area filtering
- HUD visualization with FPS monitoring
- Trajectory tail visualization

YOLOv8s was selected because it provides a strong balance between accuracy, speed, and deployment feasibility. ByteTrack was selected because it maintains stable identities while remaining lightweight compared to appearance-heavy trackers. The final architecture was intentionally optimized for stable runtime performance instead of maximum complexity.

2. Small Object Detection Strategy

Aerial surveillance introduces extremely small pedestrian targets due to high-altitude drone cameras. To improve small-object detection quality, multiple optimization strategies were implemented.

- Higher inference resolution (imgsz=1024) preserved tiny-object detail.
- Dynamic confidence thresholding improved distant object recall.
- Tiny false detections were filtered using minimum box-area constraints.
- Trajectory smoothing improved temporal consistency.
- Detection scheduling improved runtime efficiency while preserving stable tracking.

3. Experimental Approaches Explored and Rejected

Several advanced approaches were explored during experimentation before finalizing the lightweight pipeline. These experiments helped evaluate tradeoffs between small-object accuracy, runtime FPS, and tracking stability.

3.1 SAHI Sliced Inference

SAHI (Sliced Aided Hyper Inference) was tested to improve tiny-object detection performance. The frame was divided into smaller slices so that tiny aerial pedestrians could occupy larger regions inside each inference patch.

- Improved far-field tiny-object recall
- Better visibility of small pedestrians
- More accurate distant detections in static scenes

However, SAHI also introduced several practical problems:

- Significant FPS reduction due to multiple inference passes
- Higher GPU usage and computational overhead
- Tracking instability during rapid drone movement
- Delayed tracker updates because sliced inference was slow
- Inconsistent detections between slices during fast camera motion

3.2 Dynamic Region of Interest (ROI)

Dynamic ROI processing was explored to allocate more computational resources to upper aerial regions where pedestrians generally appear smaller and harder to detect.

- Upper-frame regions were prioritized for higher-detail processing
- The strategy attempted to improve compute allocation efficiency
- The approach aimed to balance FPS and tiny-object recall

Despite these advantages, dynamic ROI switching introduced several challenges:

- Rapid drone motion caused unstable ROI positioning
- Objects frequently moved between ROI boundaries
- Inconsistent detections appeared during scene transitions
- Tracker stability decreased in dynamic aerial movement

After experimentation, the final submission prioritized a stable lightweight real-time architecture over aggressive sliced inference and dynamic region-adaptive processing. The final system achieved significantly better runtime stability, cleaner tracking behavior, and more consistent FPS.

4. Handling ID Switching and Ego-Motion

Drone-based tracking systems frequently suffer from ID switching because of:

- Rapid drone ego-motion
- Occlusions
- Temporary missed detections
- Scale variation
- Motion blur

To reduce ID switching and improve tracking stability, the following techniques were used:

- ByteTrack persistence preserved identities through temporary detection failures.
- Detection scheduling reduced unstable detector fluctuations.
- Trajectory smoothing reduced jitter caused by noisy detections.

- Track confirmation logic filtered short-lived false tracks.
- Dynamic thresholds improved detection consistency for distant targets.

5. Edge Hardware Deployment (NVIDIA Jetson)

The final architecture was intentionally designed to remain lightweight enough for edge-device deployment. Several optimizations can further improve deployment performance on NVIDIA Jetson hardware.

- TensorRT conversion can significantly accelerate YOLO inference.
- FP16 inference can reduce memory usage and improve throughput.
- Lower input resolution can improve edge-device FPS.
- Detection interval scheduling reduces detector execution frequency.
- YOLOv8s remains lightweight enough for Jetson-class hardware.

6. Engineering Tradeoffs

Optimization	Benefit	Tradeoff
Higher imgsz	Better tiny-object detection	Lower FPS
Detection scheduling	Higher runtime speed	Less frequent detector refresh
ByteTrack	Lightweight stable tracking	Limited appearance modeling
Trajectory smoothing	Cleaner visual tracking	Slight motion lag
SAHI inference	Improved tiny-object recall	High computational overhead
Dynamic ROI	Adaptive compute allocation	Tracking instability during motion

The final implementation focuses on balancing aerial detection quality, tracking stability, runtime efficiency, and practical deployment feasibility. Rather than maximizing architectural complexity, the project prioritizes stable real-time drone surveillance performance suitable for realistic UAV environments.